SÉANCE 1 · BASES DE DONNÉES NOSQL · 9H00 – 13H45

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose (`docker-compose-plugin`)
- Un laptop avec Docker **par groupe** suffit pour la partie pratique

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

Si les trois commandes passent, vous êtes prêts pour 10h50.

D'ici là : **papier et stylo**. Les laptops restent fermés pendant l'ice-breaker.

PLAN DE LA SÉANCE

9h00 → 13h45 : 3h de cours, puis le TP

Heure	Quoi	Mode
9h00	Accueil, brief de l'ice-breaker Zwipe	Plénière
9h10	Ice-breaker : concevoir Zwipe en SQL (20 min, papier et stylo)	Groupes de 3-4
9h30	Présentations et critiques croisées	Groupes
9h50	Debrief : là où le SQL craque, théorème CAP	Plénière
10h05	Outils vs technologies, c'est quoi NoSQL	Théorie
10h20	Pause	☕
10h30	MongoDB : cluster, serveur, base, collection, document	Théorie
10h50	Hands-on : find, filtres, insert, update, delete	Pratique, laptops
11h35	Quand choisir Mongo, quand surtout pas	Théorie + débat
11h50	Quiz et teaser séance 2	Jeu
12h00	**TP noté** : le jeu de données des restaurants de New York (`tp-1/TP.md`)	Autonomie, binômes
13h45	Fin, rendu du TP	

01 / ICE-BREAKER

Zwipe

Une app de vidéos courtes. Vous êtes l'équipe data. Vous n'avez que du SQL.

CONTEXTE

Zwipe, l'app qui a explosé cet été

- Vidéos de 6 à 180 secondes, feed infini « Pour toi »
- Lancée en juin 2026, pendant la Coupe du Monde
- **50 millions d'utilisateurs** en 4 mois
- Lives, duos, sondages, sons, produits affiliés…
- Une nouvelle fonctionnalité chaque semaine

Votre mission

L'équipe a décidé : **tout en SQL** (PostgreSQL). Proposez le schéma et les requêtes qui tiennent la charge.

Toute ressemblance avec une app existante est… vous savez laquelle.

LES 5 CAS D'USAGE À COUVRIR

Chaque groupe traite les cinq

	Cas d'usage	Contrainte
U1	**Feed « Pour toi »** : les 20 prochaines vidéos + compteur de vues qui bouge en direct	< 50 ms, 50 M de scrolls / min
U2	**Fiche vidéo** : son, duo, filtre, sous-titres, lieu, produit, sondage… et une nouvelle option par semaine	Pas deux vidéos avec les mêmes champs
U3	**« Tes amis ont aimé »** : recommandations via amis d'amis d'amis	3 niveaux, temps réel
U4	**Analytics** : vue, like, skip, replay, agrégés par heure	2 M d'événements / seconde
U5	**Live** : chat + cadeaux + classement des top donateurs	300 000 spectateurs, classement rafraîchi 10×/s

CONSIGNE

20 minutes, puis on présente

À produire

- Le **schéma** : tables, clés, index
- **Une requête SQL** par cas d'usage (pseudo-SQL accepté)
- Un point faible que vous assumez

Supports

- **Papier et stylo**, tableau blanc
- Pas de laptop, pas de téléphone, pas de ChatGPT
- **3 minutes** de présentation par groupe, feuille à la main

PENDANT QUE VOUS BOSSEZ

Timer

20:00

Appuyez sur `t` pour lancer / mettre en pause, `r` pour remettre à 20:00

PRÉSENTATIONS

Grille d'écoute pour les autres groupes

Pendant qu'un groupe présente, les autres notent : **« ça casse où ? »**

Questions à poser

- Combien de **JOIN** pour afficher un écran ?
- Que se passe-t-il quand le produit ajoute un champ **vendredi 18h** ?
- Combien de **serveurs** ? Lequel reçoit les écritures ?

Et encore

- Combien de lignes pour **amis d'amis d'amis** ?
- Que coûte **2 M INSERT / s** sur une table indexée ?
- Le classement live : **ORDER BY** sur combien de lignes, combien de fois ?

02 / DEBRIEF

Là où le SQL craque

Vos schémas étaient bons. Le problème, c'est le contexte.

LE SCHÉMA QU'ON A (PRESQUE) TOUS DESSINÉ

Version 1, lundi matin

```
CREATE TABLE users   (id, pseudo, pays, age, bio);
CREATE TABLE videos  (id, author_id, titre, duree, vues, likes);
CREATE TABLE follows (follower_id, followed_id);
CREATE TABLE likes   (user_id, video_id, ts);
CREATE TABLE events  (id, type, user_id, video_id, ts);
CREATE TABLE live_messages (id, live_id, user_id, texte, montant, ts);
```

Propre. Normalisé. Le prof de BDD relationnelle est fier.

LE SCHÉMA QU'ON A (PRESQUE) TOUS DESSINÉ

Version 7, vendredi 18h

```
CREATE TABLE users   (id, pseudo, pays, age, bio, premium_depuis, premium_plan);
CREATE TABLE videos  (id, author_id, titre, duree, vues, likes,
                      duo_avec, duo_type, lieu_nom, lieu_lat, lieu_lng,
                      sondage_question, filtre, epingle, produit_id, ...);
CREATE TABLE video_sous_titres (video_id, langue);
CREATE TABLE sondage_options   (video_id, texte, votes);
CREATE TABLE produits (id, nom, prix);
CREATE TABLE follows (follower_id, followed_id);
CREATE TABLE likes   (user_id, video_id, ts);
CREATE TABLE events  (id, type, user_id, video_id, ts, duree_vue, appareil);
CREATE TABLE live_messages (id, live_id, user_id, texte, montant, ts);
-- ALTER TABLE videos ADD COLUMN ... (x14 cette semaine)
```

Un scroll = une jointure. 50 M de scrolls / min.

Scénario. Léa ouvre Zwipe. L'app doit afficher **immédiatement** les 20 prochaines vidéos de son feed, avec le pseudo de l'auteur et le nombre de likes, et chaque visionnage incrémente le compteur de vues. 50 millions de personnes font pareil chaque minute.

```sql
SELECT v.*, u.pseudo, COUNT(l.user_id) AS likes
FROM videos v
JOIN users u  ON u.id = v.author_id
LEFT JOIN likes l ON l.video_id = v.id
WHERE v.id IN (/* 20 ids du reco */)
GROUP BY v.id, u.pseudo;

UPDATE videos SET vues = vues + 1 WHERE id = 42;
-- ×100 000 / seconde sur la même ligne
```

- La jointure est OK… une fois. Pas 800 000 fois par seconde.
- Le compteur de vues : **un verrou sur une ligne**, tout le monde fait la queue.
- Vous avez dit « on met un cache » ? Bravo. **Quel cache ?**

Indice : une structure **clé → valeur**, en mémoire, qui sait faire `INCR` en 1 µs.

40 colonnes, 35 sont NULL

Scénario. Le produit lance une option par semaine. Cette vidéo a un **sondage**, celle-là un **duo** et un **lieu**, la troisième un **produit** affilié et des sous-titres en 4 langues. Aucune n'a les mêmes champs, et la fiche complète doit s'afficher en une lecture.

```
-- option A : tout dans videos
ALTER TABLE videos ADD COLUMN sondage_question TEXT;
ALTER TABLE videos ADD COLUMN duo_avec INT;
ALTER TABLE videos ADD COLUMN duo_type VARCHAR(10);
-- … 14 ALTER plus tard, 35 colonnes NULL par ligne

-- option B : EAV (Entity-Attribute-Value)
CREATE TABLE video_attrs (video_id, cle, valeur TEXT);
-- félicitations, vous avez réinventé le JSON, en pire
```

- Chaque vidéo a un **profil différent** : duo, sondage, lieu, produit…
- `ALTER TABLE` sur 2 milliards de lignes : ça se planifie, ça ne se « push » pas.
- Le EAV : plus de typage, plus d'index utiles, des `JOIN` pour lire une seule vidéo.

Indice : et si la vidéo était stockée **telle qu'elle est lue**, un objet avec les champs qu'elle a ?

Amis d'amis d'amis = 3 auto-jointures

Scénario. Sous une vidéo, le bloc « Tes amis ont aimé ». Pour Léa, il faut remonter à ses abonnements, aux abonnements de ceux-ci, puis encore un niveau, et retrouver les vidéos likées par ces **3 niveaux**, en temps réel.

```sql
SELECT DISTINCT l.video_id
FROM follows f1
JOIN follows f2 ON f2.follower_id = f1.followed_id
JOIN follows f3 ON f3.follower_id = f2.followed_id
JOIN likes l    ON l.user_id    = f3.followed_id
WHERE f1.follower_id = 42;
-- 300 abonnements × 300 × 300 = 27 000 000 lignes intermédiaires
-- pour UN utilisateur. Ajoutez un niveau : 8 milliards.
```

- Le relationnel est bon pour les **relations fixes**, pas pour **parcourir** un réseau.
- Chaque niveau multiplie. Le plan d'exécution ressemble à un cri.
- Les CTE récursives existent. Elles ne scalent pas mieux.

Indice : un modèle où « suivre » est un **lien physique** entre deux nœuds, qu'on parcourt en O(voisins).

2 millions d'INSERT par seconde

Scénario. Chaque vue, like, skip et replay est un événement : **2 millions par seconde**. Le dashboard interne veut, par heure et par type, le nombre d'événements des dernières 24 h.

```sql
INSERT INTO events (type, user_id, video_id, ts)
VALUES ('vue', 42, 1337, now());
-- × 2 000 000 / s, un seul serveur d'écriture,
-- 4 index B-tree à maintenir à chaque ligne

SELECT date_trunc('hour', ts), type, COUNT(*)
FROM events WHERE ts > now() - interval '1 day'
GROUP BY 1, 2;
-- scan de 170 milliards de lignes. Bonne nuit.
```

- Le SQL classique écrit sur **un primaire**. Un. Les réplicas ne servent qu'à lire.
- Les index B-tree aiment les lectures, **détestent les écritures massives**.
- Agréger par heure lit 100 % des colonnes alors qu'on en veut 2.

Indice : un stockage **par colonne**, distribué sur N nœuds qui écrivent tous, trié par temps.

Top 100 donateurs, rafraîchi 10 fois par seconde

Scénario. Un live avec **300 000 spectateurs**. Le chat défile, les cadeaux (payants) tombent, et le classement des 100 plus gros donateurs s'affiche à l'écran, rafraîchi 10 fois par seconde.

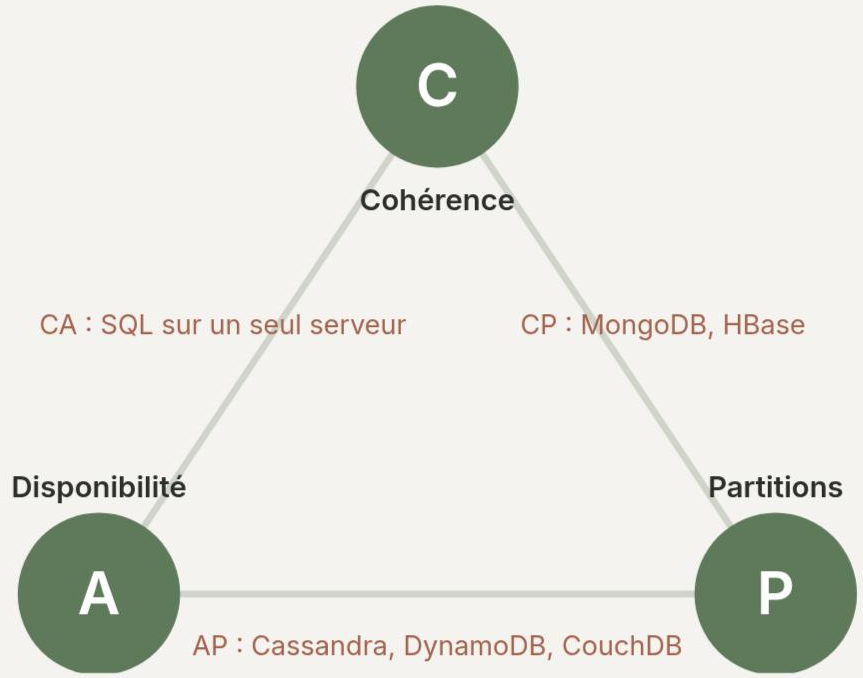
```sql
SELECT user_id, SUM(montant) AS total
FROM live_messages
WHERE live_id = 7 AND montant > 0
GROUP BY user_id
ORDER BY total DESC
LIMIT 100;
-- 300 000 spectateurs, 10×/s = 3 M agrégations/s
-- et le chat qui écrit 50 000 messages/s à côté
```

- Un classement, c'est une **structure triée maintenue**, pas une requête rejouée.
- Le chat n'a pas besoin de durabilité : il a besoin de **diffusion** (pub/sub).
- Les cadeaux, eux, doivent être durables et transactionnels. Deux besoins, deux outils.

Indice : un **sorted set** en mémoire + un canal de publication. Et une base SQL pour l'argent.

LE THÉORÈME QU'ON CITE TOUJOURS

CAP : trois promesses, on n'en tient que deux



	La promesse	Pour comprendre
C	Tout le monde lit **la même valeur** au même moment	Ton solde est identique au distributeur de Lille et à celui de Nice
A	Chaque requête reçoit **une réponse**, même pas toute fraîche	Le distributeur de Nice répond toujours, même si le siège est injoignable
P	Le système continue quand le **réseau entre serveurs** coupe	Le câble Lille–Nice est coupé, les deux agences restent ouvertes

Dès qu'on a **plusieurs serveurs**, le réseau finira par couper : P est imposé. Le vrai choix, c'est C ou A pendant la panne.

CAP EN PRATIQUE

Le câble est coupé. On fait quoi ?

Scénario. Zwipe a un serveur à Paris et un à Francfort. Le lien entre les deux tombe. Léa like une vidéo sur Paris. Au même instant, Tom demande le nombre de likes de cette vidéo à Francfort.

Choisir C : cohérence

Francfort **refuse de répondre** tant qu'il n'est pas sûr d'avoir la dernière valeur. Tom voit une erreur ou un spinner.

Outils : MongoDB (réglage par défaut), HBase, SQL avec réplication synchrone.

Bon pour : l'argent des cadeaux, un stock, un solde, une réservation.

Choisir A : disponibilité

Francfort **répond avec ce qu'il a**, peut-être vieux de 2 secondes. Tom voit 1 041 likes au lieu de 1 042. On réconcilie quand le câble revient.

Outils : Cassandra, DynamoDB, CouchDB, le DNS.

Bon pour : compteur de vues, feed, chat, panier.

Le SQL classique sur **un seul serveur** n'a pas ce problème : pas de réseau entre nœuds, donc C et A à la fois. Il n'est pas nul, il est **optimisé pour autre chose** : cohérence forte, transactions, relations stables. Mais un seul serveur ne scale pas.

RÉCAP

À chaque cas d'usage, sa famille NoSQL

Cas	Douleur SQL	Famille NoSQL	Exemple d'outil
U1 Feed	Jointures répétées, verrou sur compteur	**Clé-valeur** en mémoire	Redis, Memcached
U2 Fiche vidéo	Colonnes NULL, ALTER TABLE, EAV	**Document**	MongoDB, CouchDB
U3 Amis d'amis	Auto-jointures explosives	**Graphe**	Neo4j, Neptune
U4 Analytics	Écritures massives, un seul primaire	**Colonnes larges / séries temporelles**	Cassandra, ClickHouse, InfluxDB
U5 Live	ORDER BY rejoué, pas de diffusion	**Clé-valeur** + pub/sub	Redis (sorted sets, streams)

L'argent des cadeaux (U5), lui, reste en SQL : transactions et cohérence forte.

03 / VOCABULAIRE

Outils vs technologies

« On va utiliser du MongoDB » n'est pas une décision d'architecture.

LA SLIDE QUE TOUT LE MONDE CONFOND

Une technologie est un concept. Un outil est un produit.

Technologie (le quoi)	Outils (le avec quoi)
Stockage de documents JSON	MongoDB, CouchDB, Firestore, Amazon DocumentDB
Clé-valeur	Redis, Memcached, DynamoDB, etcd
Graphe	Neo4j, Amazon Neptune, ArangoDB
Colonnes larges	Cassandra, ScyllaDB, HBase
Séries temporelles	InfluxDB, TimescaleDB, ClickHouse (analytique colonnes)
Moteur de recherche	Elasticsearch, OpenSearch, Meilisearch
Vecteurs (IA, similarité)	Qdrant, Pinecone, pgvector, Milvus
Relationnel	PostgreSQL, MySQL, SQLite, Oracle

On choisit d'abord la technologie (le modèle de données), ensuite l'outil (licence, hébergement, équipe, prix).

MINI-QUIZ

Outil ou technologie ?

- PostgreSQL → outil (relationnel)
- SQL → ni l'un ni l'autre : un **langage**
- JSONB dans Postgres → une **technologie** (document) dans un outil relationnel. Les frontières bougent.
- Kafka → outil, technologie « journal d'événements distribué ». Pas une base de données.
- Firebase → une **plateforme** qui contient un outil document (Firestore)
- « Le cloud » → un ordinateur de quelqu'un d'autre

DÉFINITION

NoSQL = Not only SQL

- Pas « pas de SQL » : **pas seulement** SQL. Le mot vient d'un hashtag de meetup à San Francisco, juin 2009.
- Point commun des familles : on **renonce** à quelque chose du relationnel pour **gagner** autre chose.

On renonce à	On gagne	Exemple Zwipe
Un **schéma imposé**	De la flexibilité	La vidéo avec sondage et celle avec duo vivent dans la même collection, sans `ALTER TABLE`
Les **jointures**	Des lectures locales rapides	La fiche vidéo (auteur, son, sondage) est lue en **un seul** document
Les **transactions globales**	Le scaling horizontal	Les 2 M d'événements / s s'écrivent sur 20 nœuds en parallèle, sans verrou commun
Un **langage standard** (SQL)	Un modèle adapté au problème	Amis d'amis d'amis en 3 lignes de Cypher (Neo4j) au lieu de 3 auto-jointures

LES FAMILLES

4 historiques, 2 nouvelles

Clé-valeur

Un dictionnaire géant. Ultra rapide, zéro requête complexe.

`Redis`

Document

Des objets JSON imbriqués, requêtables par champ.

`MongoDB`

Colonnes larges

Des lignes avec des millions de colonnes, réparties par clé.

`Cassandra`

Graphe

Nœuds + arêtes. Parcourir plutôt que joindre.

`Neo4j`

Recherche

Index inversé, texte intégral, pertinence.

`Elasticsearch`

Vecteurs

Similarité sémantique, RAG, « montre-moi des vidéos comme celle-là ».

`Qdrant`

LE VRAI CHANGEMENT DE MENTALITÉ

Schema-on-write vs schema-on-read

Relationnel : schema-on-write

La base **refuse** ce qui ne rentre pas dans les colonnes. Le contrat est vérifié à l'écriture.

Le videur de boîte : pas de baskets, pas d'entrée.

Document : schema-on-read

La base **accepte** à peu près tout. C'est le code qui lit qui doit gérer les variantes.

Le festival : tout le monde entre, c'est toi qui gères ce que tu trouves dans la tente.

Flexible ≠ sans schéma. Le schéma existe toujours : il a juste déménagé **dans votre code**. Et si personne ne le documente, il est dans la tête de celui qui a démissionné.

ÉCHELLE

Verticale vs horizontale

Verticale : un plus gros serveur

Plus de RAM, plus de CPU. Simple. Jusqu'au jour où le plus gros serveur du catalogue ne suffit plus, ou coûte un rein.

Une seule 4L, moteur de Ferrari.

Horizontale : plus de serveurs

On découpe les données (sharding) sur N machines banales. Les NoSQL sont nés pour ça.

Un essaim de trottinettes. Chacune porte un bout.

Prix à payer : les **jointures entre machines** deviennent impossibles ou très chères. D'où « ce qui est lu ensemble est stocké ensemble ».

Les frontières bougent

- PostgreSQL fait du **JSONB** indexé, des vecteurs (`pgvector`), du géo. « Just use Postgres » est devenu un meme à part entière.
- MongoDB fait des **transactions multi-documents** depuis 4.0, du texte intégral, des vecteurs (Atlas Vector Search).
- Redis sait faire du JSON, des graphes, des séries temporelles via modules.
- Conclusion : on ne choisit plus « SQL ou NoSQL ». On choisit **un modèle de données principal** et on accepte des compromis sur le reste.

04 / MONGODB

MongoDB, de haut en bas

Cluster → serveur → base → collection → document → champ

CARTE D'IDENTITÉ

MongoDB

- **2009**, société 10gen (devenue MongoDB Inc.)
- Le nom vient de humongous (« énorme »)
- Modèle : **document**, stocké en BSON (JSON binaire)
- Langage : pas SQL, une API de requêtes en objets JSON
- Licence SSPL (pas tout à fait open source), version cloud **Atlas**
- Version du cours : **7.0**

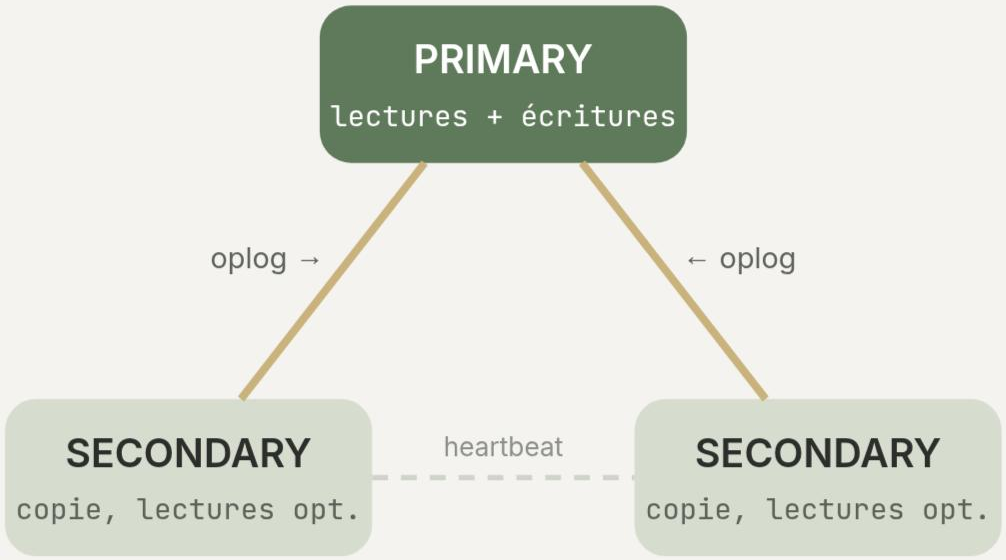
Ce qu'il fait bien

Schéma flexible, lectures d'objets entiers, scaling horizontal natif, réplication intégrée, index riches (texte, géo, TTL).

Ce qu'il fait moins bien

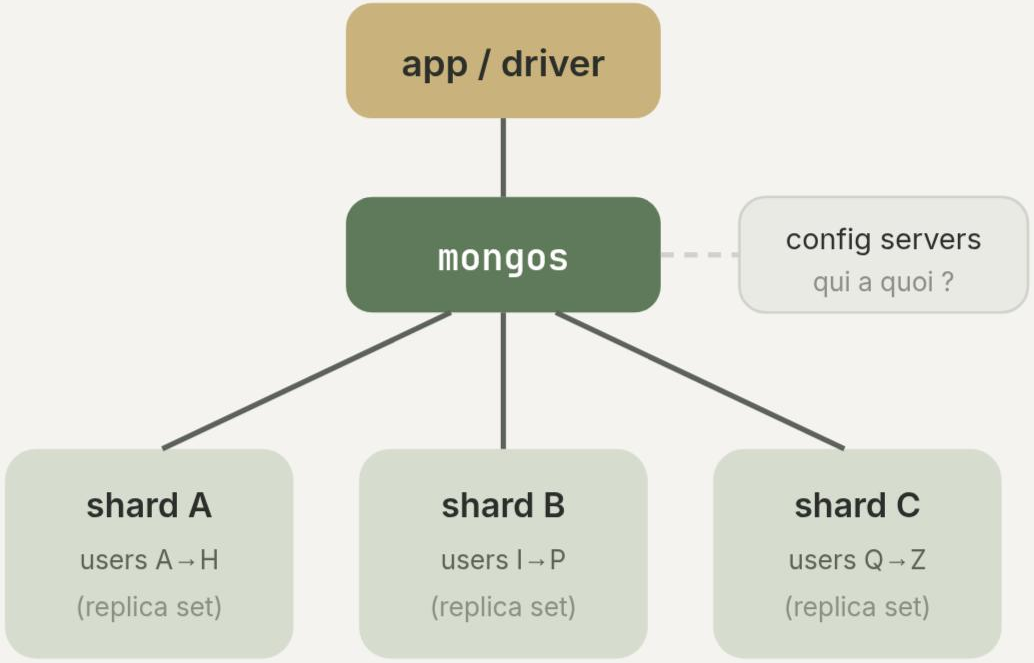
Jointures, transactions complexes multi-entités, reporting relationnel.

Replica set : la haute disponibilité



- Minimum **3 nœuds**. Un primaire reçoit les écritures, les secondaires rejouent son journal (oplog).
- Le primaire tombe ? **Élection** en quelques secondes, un secondaire prend le relais. L'app ne voit (presque) rien.
- On peut lire sur les secondaires, au prix d'un léger retard. C'est le **C vs A** de tout à l'heure, réglable par requête.

Sharding : la montée en charge



- Les données sont **découpées par une clé de shard** (ex : `auteur`) sur plusieurs replica sets.
- `mongos` est le routeur : il sait, grâce aux config servers, quel shard interroger.
- Bonne clé de shard = requêtes qui touchent **un seul shard**. Mauvaise clé = toutes les requêtes partout = plus lent qu'avant.
- Pour ce cours : **un seul mongod**. Le sharding, c'est quand vous aurez 50 M d'utilisateurs.

Inclus d'office, mais pas automatique

Ce qui est dans la boîte

Replica set et sharding **font partie de MongoDB**, édition Community comprise. Aucune extension, aucun plugin, aucun logiciel tiers : c'est le même binaire `mongod`.

Ce qui ne se fait pas tout seul

Un `mongod` démarre **seul (standalone)**, comme dans notre Docker. C'est à vous de lancer 3 processus et de les déclarer avec `rs.initiate()`, puis d'ajouter `mongos`, les config servers et `sh.shardCollection()` pour sharder.

- Une fois configuré, le reste est **automatique** : élection d'un nouveau primaire, réplication via l'oplog, équilibrage des chunks entre shards.
- **Atlas**, le cloud de MongoDB, fait cette configuration pour vous : un curseur « 3 nœuds », un bouton « sharder ». Payant, mais c'est là que la plupart des équipes finissent.
- En résumé : par défaut, un serveur seul. Par conception, un cluster à portée de trois commandes.

mongod, mongosh, drivers

`mongod`

Le **serveur**. Un processus, un port (27017), un dossier de données. Moteur de stockage **WiredTiger** : compression, cache en RAM, journal pour la durabilité.

`mongosh`

Le **shell**. Un REPL JavaScript branché sur le serveur. C'est ce qu'on utilise aujourd'hui. `db` est la base courante.

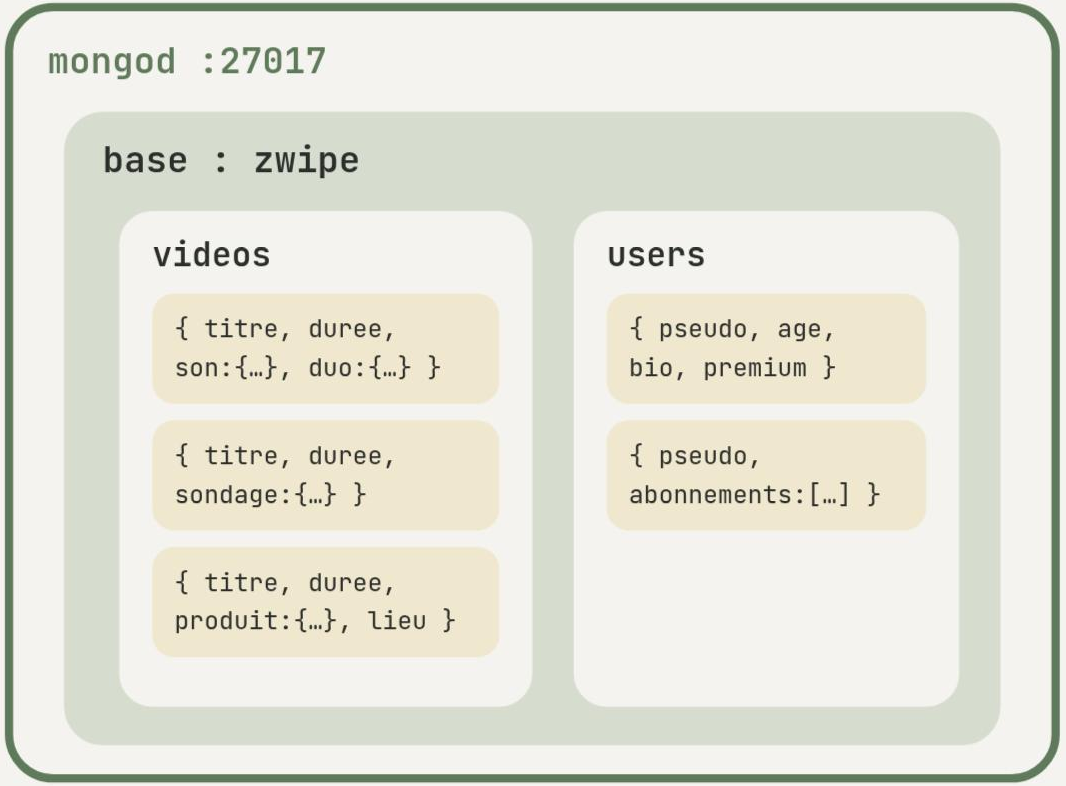
Drivers

Ce que votre **application** utilise : `pymongo` (Python), `mongodb` (Node), `mongoose` (ODM Node). Même API que le shell, à peu de choses près.

Et **Compass** : l'interface graphique officielle. Pratique pour explorer, mais apprenez d'abord le shell, c'est ce qui finit dans le code.

NIVEAU 3 À 6 · LA HIÉRARCHIE

Serveur → base → collection → document → champ



MongoDB	≈ SQL	Mais
Serveur	Serveur	
Base	Base / schéma	créée à la 1re écriture
Collection	Table	**sans contrat** de colonnes
Document	Ligne	imbriqué, tableaux
Champ	Colonne	peut manquer, changer de type
Index	Index	pareil, + texte, géo, TTL
`$lookup`	JOIN	à éviter

JSON à l'écran, BSON sur le disque

```
{
  _id: ObjectId('000000000000000000003e9'),
  auteur: ObjectId('000000000000000000002b'),
  titre: "Micro-trottoir : c'est quoi NoSQL ?",
  duree: 105,
  vues: 1958516,
  hashtags: [ '#lego', '#fitcheck', '#recette' ],
  son: { titre: 'Night Shift', artiste: 'DJ Mongo' },
  publieLe: ISODate('2026-05-28T16:00:00.000Z'),
  produit: { nom: 'Hoodie oversize', prix: 59.99 }
}
```

BSON = Binary JSON. Même forme, mais avec des types que JSON n'a pas :

- `ObjectId`, `Date`, `Int32` / `Long`, `Decimal128` (pour l'argent !), `Binary`
- Tableaux, sous-documents, à volonté (100 niveaux max)

Taille max d'un document : **16 Mo**. Si vous l'atteignez, votre modèle est faux, pas la limite.

NIVEAU 5 · LE DOCUMENT

L'`_id` et l'ObjectId

Chaque document a un `_id` **unique et obligatoire**. Si vous ne le donnez pas, Mongo génère un `ObjectId` : 12 octets.

```
6ae7c7af 388928108d 8d6ae5
   │          │        └─ compteur (3 octets)
   │          └────────── aléa par processus (5 octets)
   └─────────────────────── timestamp Unix (4 octets)
```

Donc : **trié par date de création**, et générable côté client sans demander au serveur.

Dans mongosh

```
ObjectId('6ae7c7af388928108d8d6ae5').getTimestamp()
// ISODate('2026-11-02T...')
```

Dans le seed du cours, les `_id` sont forcés (`…0001` , `…03e9`) pour que tout le monde ait les mêmes. En vrai, laissez Mongo faire.

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

NoSQL : quand les tableaux ne suffisent plus

Pas de panique, SQL n'est pas mort. Il est juste fatigué.

Badmavasan KIROUCHENASSAMY

Ice-breaker « Zwipe » · Outils vs technologies · MongoDB de A à `$lookup`

AVANT DE COMMENCER

Il vous faut Docker

Installer

- **Windows / macOS** : Docker Desktop (docs.docker.com/get-docker)
- **Linux** : Docker Engine + le plugin Compose

Vérifier

```
docker --version
docker compose version
docker pull mongo:7.0      # télécharge l'image dès maintenant
```

CORRECTION

On corrige ensemble

Un groupe au clavier par exercice. Les autres jugent.

Les solutions complètes sont dans `lab/solutions.md`, disponibles après la séance.

06 / DÉCIDER

Quand choisir Mongo. Quand surtout pas.

SPÉCIFICATIONS À CONNAÎTRE

Les chiffres qui tombent à l'oral

Limite	Valeur
Taille d'un document	**16 Mo**
Profondeur d'imbrication	100 niveaux
Index par collection	64
Champs dans un index composé	32
Replica set	max 50 membres, 7 votants
Port par défaut	27017

Transactions multi-documents : depuis 4.0 (replica set) et 4.2 (sharded). ACID, mais plus lentes et limitées à 60 s par défaut. Un bon modèle en a rarement besoin.

writeConcern : combien de nœuds doivent confirmer l'écriture (`majority` = sûr). **readConcern** : quelle fraîcheur on accepte en lecture. C'est **CAP réglable par requête**.

Validation de schéma : possible (`$jsonSchema`) et recommandée dès que l'équipe dépasse 2 personnes.

CHOISIR MONGO QUAND

Le feu vert

- **Le schéma évolue vite** et varie d'un objet à l'autre : catalogue produit, fiche vidéo, profil, CMS, formulaires.
- **On lit des objets entiers**, rarement des morceaux croisés : « donne-moi cette vidéo avec tout ce qu'elle contient ».
- **Il faut scaler horizontalement** sans réécrire l'app : sharding intégré, réplication intégrée.
- **Prototype rapide** : pas de migration pour chaque idée. Le hackathon, la startup, le MVP.
- **Données semi-structurées** : logs applicatifs, événements IoT légers, imports d'API JSON.
- L'équipe code en JS/Python et pense **en objets**, pas en tables.

SURTOUT PAS QUAND

Le feu rouge

- **Les données sont fortement relationnelles** : comptabilité, ERP, facturation, stocks. Beaucoup d'entités, beaucoup de liens, beaucoup de rapports croisés.
- **Transactions complexes** sur plusieurs entités à chaque opération : virement bancaire, réservation multi-ressources.
- **Reporting lourd** avec jointures à volonté pour la finance ou le contrôle de gestion. Ils veulent du SQL, donnez-leur du SQL.
- **Votre modèle a besoin de `$lookup` partout.** Signal clair : vous avez normalisé une base document. Vous vouliez Postgres.
- **Volume d'événements massif** (U4) : Mongo tient, mais une base colonnes ou séries temporelles fait mieux pour 5 fois moins cher.
- Cache, compteurs, classements, sessions (U1, U5) : **Redis**, pas Mongo.

ANTI-PATTERNS

Les 5 façons de rater Mongo

1 · Le tableau sans fin

Embarquer tous les commentaires dans la vidéo. Le document grossit, dépasse 16 Mo, chaque lecture charge 2 millions de commentaires.

2 · Le document de 15,9 Mo

« On est encore sous la limite. » Non. Si vous surveillez la limite, le modèle est faux.

3 · Le SQL en Mongo

Une collection par entité, un `$lookup` par relation. Vous avez le pire des deux mondes.

4 · L'`_id` métier

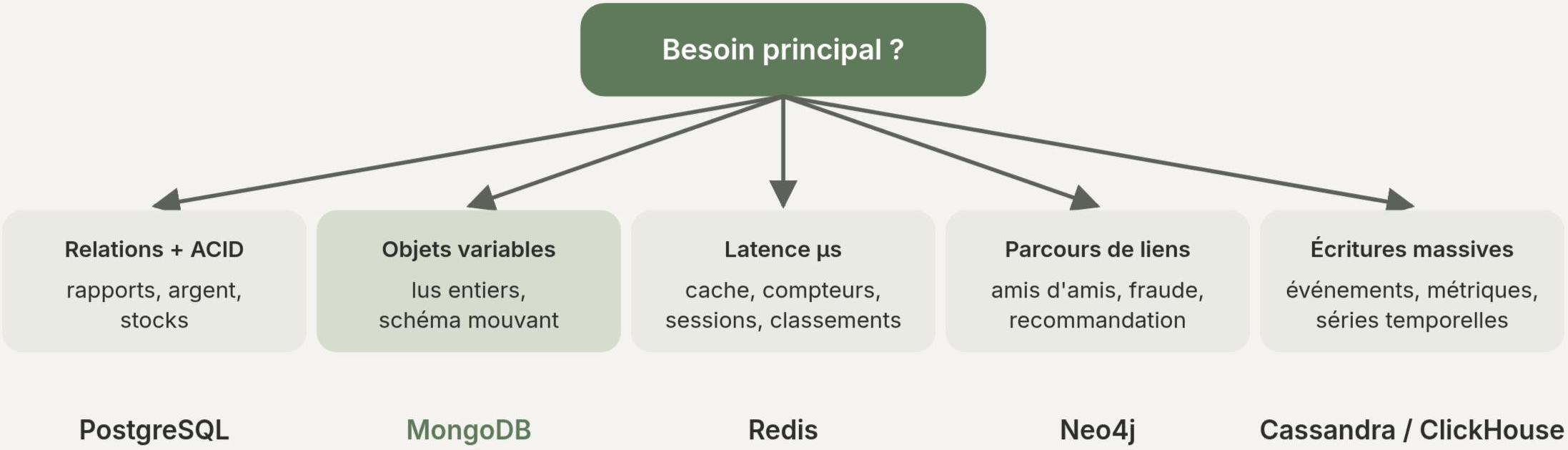
Mettre l'email en `_id`. Jusqu'au jour où l'utilisateur change d'email. (Il va changer.)

5 · Zéro index, zéro validation

« C'est flexible. » Flexible ne veut pas dire « lent et incohérent ». `explain()` et `$jsonSchema` existent.

ARBRE DE DÉCISION

En une slide



« Je ne sais pas » → PostgreSQL (avec JSONB), puis on verra. Le choix se fait par usage, pas par application.

Recherche plein texte → Elasticsearch · Similarité IA → base vectorielle (ou pgvector / Atlas)

RETOUR SUR ZWIPE

L'architecture qu'on aurait dû proposer

Cas	Techno	Outil	Et Mongo ?
U1 Feed + compteurs	Clé-valeur mémoire	Redis (`INCR`, listes)	stocke les vidéos derrière le cache
U2 Fiche vidéo	Document	**MongoDB**	**oui, c'est son job**
U3 Amis d'amis	Graphe	Neo4j	non (les abonnements en tableau suffisent à 1 niveau)
U4 Analytics	Colonnes / séries temporelles	ClickHouse ou Cassandra	possible à petite échelle, pas à 2 M/s
U5 Live	Clé-valeur + pub/sub	Redis (sorted sets, streams)	non
Paiements, cadeaux	Relationnel	PostgreSQL	non, jamais, l'argent c'est ACID

Plusieurs technologies pour une app : c'est la « persistance polyglotte ». Autant de choses à maintenir, c'est le prix.

07 / CLÔTURE

Quiz, à retenir, suite

QUIZ · 8 QUESTIONS, RÉPONSE AU CLIC

Partie 1

1. NoSQL veut dire... **Not only SQL**
2. MongoDB est un outil ou une technologie ? **Un outil** (techno : base document)
3. L'équivalent d'une table en Mongo ? **Une collection**
4. Taille max d'un document ? **16 Mo**
5. `{ age: { $ne: 18 } }` matche-t-il un document sans champ `age` ? **Oui.** L'absence n'est pas 18.

QUIZ

Partie 2

6. `updateOne({…}, { titre: "x" })` sans `$set`, que se passe-t-il ? **Erreur.** Un update exige des opérateurs. `replaceOne` sinon.
7. Le compteur de vues d'une vidéo virale : Mongo `$inc` ou Redis `INCR` ? **Redis** en frontal, Mongo synchronisé derrière. `$inc` est atomique mais écrit sur disque.
8. Pourquoi un feed infini n'utilise-t-il jamais `skip()` ? **Il lit et jette tout ce qu'il saute.** Pagination par curseur.

Bonus : « On va stocker les paiements dans Mongo parce que c'est flexible. » Qu'est-ce qu'on répond ? Non.

À RETENIR

Cinq phrases

1. Le SQL n'a pas échoué : il est fait pour les relations stables et les transactions, pas pour tout.
2. On choisit une **technologie** (modèle de données) avant un **outil**.
3. Mongo = documents JSON, schéma flexible, **ce qui est lu ensemble est stocké ensemble**.
4. Le schéma n'a pas disparu, il a déménagé dans votre code. Documentez-le, validez-le.
5. Un `$lookup` de temps en temps, d'accord. Partout : vous vouliez Postgres.

SÉANCE 2

La suite

Agrégation avancée

`$facet`, `$bucket`, fenêtres, `$merge`. Le vrai moteur analytique de Mongo.

Redis

On construit U1 et U5 pour de vrai : compteurs, classement, pub/sub. En 40 lignes.

Neo4j

« Tes amis ont aimé » en une requête Cypher de 3 lignes. Et pourquoi elle est rapide.

À faire d'ici là : terminer `lab/exercices.md`, et **modéliser votre propre app** (une page : collections, ce qui est embarqué, ce qui est référencé).

RESSOURCES

Pour creuser

- Documentation MongoDB : mongodb.com/docs/manual (opérateurs de requête, modélisation)
- MongoDB University : cours gratuits, certification « Associate Developer »
- Martin Kleppmann, Designing Data-Intensive Applications : le livre de chevet de tout ce cours
- db-engines.com/ranking : le paysage des bases, mis à jour chaque mois
- Ce dépôt : `index.html`, `lab/`, `animateur.md`

Merci. Allez scroller.